

IPL CRICKET ANALYTICS

SQL QUERIES & EXPLANATIONS

Sections A–M

Prepared from the uploaded IPL SQL Analytics PDFs

This document collects Sections A through M from the two uploaded IPL Cricket Analytics PDFs. For each scenario it gives the question, the SQL query printed in the source, and a short explanation based on the source's strategy/interpretation. Sections J1 and J2 are not present in the supplied student booklet, so they are not invented.

SQL Reference Used in the Booklet

- v_ball — one row represents one ball.
- matches_clean — one row represents one match.
- v_match_totals — one row represents one finished match with first-innings and chase information.
- v_innings — one row represents one team's innings.
- Strike rate = $(\text{runs} \div \text{legal balls}) \times 100$.
- Economy rate = $(\text{runs charged to the bowler} \div \text{legal balls}) \times 6$.

Section A — Batters — the basics

A1 — The team wants a one-page list of the highest run-scorers in IPL history to put on the wall.

Source interpretation: V Kohli leads with 9,040 runs — more than 1,800 clear of the next name. A gap that large is worth checking rather than assuming; it holds up.

SQL Query:

```
SELECT batter, SUM(batter_runs) AS runs -- total runs
FROM v_ball -- one row = one ball
GROUP BY batter -- one row per group
ORDER BY runs DESC -- sort
LIMIT 5;
```

A2 — A sponsor will pay per six hit. Which batter should they sign?

Source interpretation: CH Gayle has 357 sixes, 46 more than the next batter, and he is not in the top three run-scorers. Volume and power are different questions.

SQL Query:

```
SELECT batter, -- what to show
SUM(CASE WHEN batter_runs = 6 THEN 1 -- sixes
ELSE 0 END) AS sixes -- branches of the CASE
FROM v_ball -- one row = one ball
GROUP BY batter -- one row per group
ORDER BY sixes DESC -- sort
LIMIT 5;
```

A3 — Coach wants the fastest scorers, but only counting batters who have actually played a lot.

Source interpretation: AD Russell strikes at 176.15 across 1,505 balls. The 1,000-ball minimum is what makes this list mean anything.

SQL Query:

```
SELECT batter, -- what to show
SUM(is_legal) AS balls, -- sample size
SUM(batter_runs) AS runs, -- total runs
ROUND(100.0*SUM(batter_runs)/SUM(is_legal),2) -- runs per 100 bal
AS strike_rate
FROM v_ball -- one row = one ba
GROUP BY batter -- one row per group
HAVING balls >= 1000 -- filter the groups
ORDER BY strike_rate DESC -- sort
LIMIT 5;
```

A4 — Show what happens when you forget the minimum-balls rule.

Source interpretation: KT Maphaka strikes at 400 — from 2 balls. This is the same query as A3 without the minimum, and it is nonsense.

SQL Query:

```
SELECT batter, -- what to show
SUM(is_legal) AS balls, -- sample size
ROUND(100.0*SUM(batter_runs)/SUM(is_legal),2) -- runs per 100 bal
AS strike_rate
FROM v_ball -- one row = one ba
GROUP BY batter -- one row per group
HAVING balls >= 1 -- filter the groups
ORDER BY strike_rate DESC -- sort
LIMIT 5;
```

A5 — Which batter scores the highest share of runs in boundaries?

Source interpretation: AD Russell takes 78.54% of his runs in boundaries. A batter that dependent on the rope is a different risk from one who rotates strike.

SQL Query:

```
SELECT batter, -- what to show
SUM(batter_runs) AS runs, -- total runs
ROUND(100.0*SUM(CASE WHEN batter_runs -- round it off
IN (4,6) THEN batter_runs ELSE
0 END) /SUM(batter_runs),2) AS -- total runs
pct_from_boundaries
FROM v_ball -- one row = one ball
GROUP BY batter -- one row per group
HAVING runs >= 2000 -- filter the groups
ORDER BY pct_from_boundaries DESC -- sort
```

```
LIMIT 5;
```

A6 — Who survives longest? Batting average, not strike rate.

Source interpretation: B Sai Sudharsan averages 47.13, ahead of KL Rahul and Warner. Average rewards survival; strike rate rewards speed. They rank people differently.

SQL Query:

```
SELECT b.batter, -- what to show
SUM(b.batter_runs) AS runs,
(SELECT COUNT(*) FROM v_ball x WHERE -- row count
x.player_out = b.batter) AS
outs,
ROUND(1.0*SUM(b.batter_runs)/ (SELECT -- round it off
COUNT(*) FROM v_ball x WHERE -- row count
x.player_out = b.batter),2) AS
average
FROM v_ball b -- one row = one ball
GROUP BY b.batter -- one row per group
HAVING runs >= 2000 -- filter the groups
ORDER BY average DESC -- sort
LIMIT 5;
```

A7 — Which batter plays the most dot balls, as a share of balls faced?

Source interpretation: SC Ganguly plays a dot ball 47.26% of the time. Nearly half of his deliveries produced nothing.

SQL Query:

```
SELECT batter, -- what to show
SUM(is_legal) AS balls, -- sample size
ROUND(100.0*SUM(CASE WHEN batter_runs -- round it off
= 0 AND is_legal = 1 THEN 1
ELSE 0 END) /SUM(is_legal),2) -- sample size
AS dot_pct
FROM v_ball -- one row = one ball
GROUP BY batter -- one row per group
HAVING balls >= 1000 -- filter the groups
ORDER BY dot_pct DESC -- sort
LIMIT 5;
```

A8 — Left-handers or right-handers — who scores faster overall?

Source interpretation: Left-handers strike at 135.10 and right-handers at 132.51 — a gap of about 2.6. Small, but measured on 277,000 balls.

SQL Query:

```
SELECT batsman_type, -- what to show
SUM(is_legal) AS balls, -- sample size
SUM(batter_runs) AS runs, -- total runs
ROUND(100.0*SUM(batter_runs)/SUM(is_legal),2) -- runs per 100 bal
AS strike_rate
FROM v_ball -- one row = one ba
GROUP BY batsman_type;
```

Section B — Batting roles — who suits which slot

B1 — Find an ideal middle-order batter who can lift the run rate in overs 7 to 15.

Source interpretation: V Sehwag strikes at 173.81 in the middle overs, from 611 balls. That is the brief from 3.1, answered.

SQL Query:

```
SELECT batter, -- what to show
  SUM(is_legal) AS balls, -- sample size
  SUM(batter_runs) AS runs, -- total runs
  ROUND(100.0*SUM(batter_runs)/SUM(is_legal),2) -- runs per 100 bal
  AS sr_middle
FROM v_ball -- one row = one ba
WHERE phase = 'Middle' -- middle overs only
GROUP BY batter -- one row per group
HAVING balls >= 500 -- filter the groups
ORDER BY sr_middle DESC -- sort
LIMIT 5;
```

B2 — Find a finisher: highest strike rate in the last five overs.

Source interpretation: AB de Villiers strikes at 225.33 in the last five overs, from 829 balls. That is the highest number in this entire step.

SQL Query:

```
SELECT batter, -- what to show
  SUM(is_legal) AS balls, -- sample size
  ROUND(100.0*SUM(batter_runs)/SUM(is_legal),2) -- runs per 100 bal
  AS sr_death
FROM v_ball -- one row = one ba
WHERE phase = 'Death' -- death overs only
GROUP BY batter -- one row per group
HAVING balls >= 300 -- filter the groups
ORDER BY sr_death DESC -- sort
LIMIT 5;
```

B3 — Find a powerplay opener who attacks from ball one.

Source interpretation: Abhishek Sharma strikes at 172.98 in the powerplay across 744 balls, ahead of SP Narine at 170.64.

SQL Query:

```
SELECT batter, -- what to show
  SUM(is_legal) AS balls, -- sample size
  ROUND(100.0*SUM(batter_runs)/SUM(is_legal),2) -- runs per 100 bal
  AS sr_pp
FROM v_ball -- one row = one ba
WHERE phase = 'Powerplay' -- powerplay only
GROUP BY batter -- one row per group
HAVING balls >= 600 -- filter the groups
ORDER BY sr_pp DESC -- sort
LIMIT 5;
```

B4 — Find an anchor: someone who scores steadily and rarely gets out.

Source interpretation: JP Duminy averages 39.78 at a strike rate of 124.40. The filter here defines the role — a strike rate band, not a maximum.

SQL Query:

```
SELECT b.batter, -- what to show
  SUM(b.batter_runs) AS runs,
  ROUND(100.0*SUM(b.batter_runs)/SUM(b.is_legal),2) -- round it off
  AS sr,
  ROUND(1.0*SUM(b.batter_runs)/ (SELECT -- round it off
    COUNT(*) FROM v_ball x WHERE -- row count
    x.player_out = b.batter),2) AS
  average
FROM v_ball b -- one row = on
GROUP BY b.batter -- one row per
HAVING runs >= 2000 AND sr BETWEEN 115 AND -- filter the g
132
ORDER BY average DESC -- sort
LIMIT 5;
```

B5 — Which batters change gear the most between the powerplay and the death overs?

Source interpretation: AB de Villiers goes from 119.14 in the powerplay to 225.33 at the death — he nearly doubles his rate. That is what "changing gear" means, measured.

SQL Query:

```
SELECT batter, -- what to show
  ROUND(100.0*SUM(CASE WHEN -- round it off
```

```

phase='Powerplay' THEN
batter_runs ELSE 0 END)
/NULLIF(SUM(CASE WHEN -- divide, no zero risk
phase='Powerplay' THEN
is_legal ELSE 0 END),0),2) AS
sr_pp,
ROUND(100.0*SUM(CASE WHEN -- round it off
phase='Death' THEN batter_runs
ELSE 0 END) /NULLIF(SUM(CASE -- branches of the CASE
WHEN phase='Death' THEN
is_legal ELSE 0 END),0),2) AS
sr_death
FROM v_ball -- one row = one ball
GROUP BY batter -- one row per group
HAVING SUM(CASE WHEN phase='Powerplay' THEN -- filter the groups
is_legal ELSE 0 END) >= 300 AND
SUM(CASE WHEN phase='Death' THEN -- if-then-else per row
is_legal ELSE 0 END) >= 300
ORDER BY (sr_death - sr_pp) DESC -- sort
LIMIT 5;

```

B6 — Who is the most reliable in a chase — strike rate batting second?

Source interpretation: V Sehwag strikes at 164.21 when chasing, from 869 balls. Chasing is a different job from setting a total and this separates the two.

SQL Query:

```

SELECT batter, -- what to show
SUM(is_legal) AS balls, -- sample size
ROUND(100.0*SUM(batter_runs)/SUM(is_legal),2) -- runs per 100 bal
AS sr_chasing
FROM v_ball -- one row = one ba
WHERE innings = 2 -- second innings o
GROUP BY batter -- one row per group
HAVING balls >= 800 -- filter the groups
ORDER BY sr_chasing DESC -- sort
LIMIT 5;

```

B7 — Compare a named anchor and a named attacker on the same three measures.

Source interpretation: Russell: 176.15 strike rate, 39.60% dots. Kohli: 134.32 strike rate, 32.82% dots. Two completely different jobs, both valuable.

SQL Query:

```

SELECT batter, -- what to show
SUM(is_legal) AS balls, -- sample size
SUM(batter_runs) AS runs, -- total runs
ROUND(100.0*SUM(batter_runs)/SUM(is_legal),2) -- runs per 100 bal
AS sr,
ROUND(100.0*SUM(CASE WHEN -- round it off
batter_runs=0 AND is_legal=1
THEN 1 ELSE 0 -- branches of the
END)/SUM(is_legal),2) AS -- sample size
dot_pct
FROM v_ball -- one row = one ba
WHERE batter IN ('V Kohli','AD Russell') -- these rows only
GROUP BY batter;

```

B8 — How many balls does a batter usually need before they start scoring quickly?

Source interpretation: Batters strike at 116.75 in their first ten balls, 144.67 in the next ten and 152.87 after that. Settling in is real and it is measurable.

SQL Query:

```

SELECT batter, match_id, -- name a first query
innings, batter_runs, is_legal,
ROW_NUMBER() OVER (PARTITION BY -- numbers rows in group
batter, match_id, innings ORDER BY
over_number, ball_number) AS ball_no
FROM v_ball WHERE is_legal = 1) -- one row = one ball
SELECT CASE WHEN ball_no <= 10 THEN '1-10 -- if-then-else per row
balls' WHEN ball_no <= 20 THEN '11-20
balls' ELSE '21+ balls' END AS
innings_stage,
ROUND(100.0*SUM(batter_runs)/COUNT(*),2) -- total runs
AS strike_rate
FROM numbered -- the source table
GROUP BY innings_stage -- one row per group
ORDER BY innings_stage;

```

B9 — Which batters are hardest to bowl a dot ball to in the death overs?

Source interpretation: JP Duminy is dotted up only 20.16% of the time at the death. A low dot rate at the death is worth as much as a high strike rate.

SQL Query:

```
SELECT batter, -- what to show
       SUM(is_legal) AS balls, -- sample size
       ROUND(100.0*SUM(CASE WHEN -- round it off
batter_runs=0 AND is_legal=1
       THEN 1 ELSE 0 -- branches of the CASE
       END)/SUM(is_legal),2) AS -- sample size
       dot_pct
FROM v_ball -- one row = one ball
WHERE phase='Death' -- death overs only
GROUP BY batter -- one row per group
HAVING balls >= 300 -- filter the groups
ORDER BY dot_pct ASC -- sort
LIMIT 5;
```

Section C — Phases — where the innings is won

C1 — How do runs and wickets differ across the three phases of an innings?

Source interpretation: The death overs cost 10.04 runs an over against 8.03 in the powerplay, and the wicket rate more than doubles from 4.01% to 8.46%. Runs and risk rise together.

SQL Query:

```
SELECT phase, -- what to show
       SUM(is_legal) AS balls, -- sample size
       ROUND(6.0*SUM(total_runs)/SUM(is_legal),2) -- runs per over
       AS runs_per_over,
       ROUND(100.0*SUM(is_wicket)/SUM(is_legal),2) -- wicket chance per
       AS wicket_pct
FROM v_ball -- one row = one ball
GROUP BY phase;
```

C2 — What is the average score at the end of the powerplay?

Source interpretation: The average powerplay score is 48.1. That single number is the benchmark a commentator is using when they say a team is "ahead of the rate".

SQL Query:

```
SELECT ROUND(AVG(pp_runs),1) AS -- what to show
       avg_powerplay_score
FROM ( SELECT match_id, innings, -- the source table
       SUM(total_runs) AS pp_runs FROM
       v_ball WHERE phase='Powerplay' AND
       innings IN (1,2) GROUP BY match_id,
       innings);
```

C3 — Does a big powerplay actually lead to a big total?

Source interpretation: A powerplay under 40 leads to an average total of 140.1. A powerplay of 70 or more leads to 198.7. A good start is worth roughly 60 runs.

SQL Query:

```
SELECT match_id, innings, -- name a first query
       SUM(total_runs) AS pp_runs FROM
       v_ball WHERE phase='Powerplay' GROUP
       BY match_id, innings), tot AS (SELECT
       match_id, innings, SUM(total_runs) AS
       total_runs FROM v_ball GROUP BY
       match_id, innings)
SELECT CASE WHEN pp.pp_runs < 40 THEN 'Under -- if-then-else per row
40' WHEN pp.pp_runs < 55 THEN '40-54'
WHEN pp.pp_runs < 70 THEN '55-69'
ELSE '70+' END AS powerplay_band, -- branches of the CASE
COUNT(*) AS innings_count, -- row count
ROUND(AVG(tot.total_runs),1) AS -- round it off
avg_final_total
FROM pp -- the source table
JOIN tot ON tot.match_id=pp.match_id AND -- attach the other table
tot.innings=pp.innings
GROUP BY powerplay_band -- one row per group
ORDER BY avg_final_total;
```

C4 — Which over of the innings is the most expensive?

Source interpretation: The 20th over costs 11.39 runs — nearly twice the first over. The last five overs are the five most expensive of the innings, in order.

SQL Query:

```
SELECT over_number + 1 AS over_no, -- what to show
       ROUND(6.0*SUM(total_runs)/SUM(is_legal),2) -- runs per over
       AS runs_per_over
FROM v_ball -- one row = one ball
GROUP BY over_number -- one row per group
ORDER BY runs_per_over DESC -- sort
LIMIT 5;
```

C5 — Which over sees the most wickets fall?

Source interpretation: 1,433 wickets fall in the 20th over, far more than any other. Batters accept the risk because there is nothing left to save wickets for.

SQL Query:

```
SELECT over_number + 1 AS over_no, -- what to show
       SUM(is_wicket) AS wickets -- wickets
FROM v_ball -- one row = one ball
```

```
GROUP BY over_number -- one row per group
ORDER BY wickets DESC -- sort
LIMIT 5;
```

C6 — How many runs are scored in the last over compared with the first?

Source interpretation: The first over goes at 6.28 and the last at 11.39. Same six balls, nearly double the runs.

SQL Query:

```
SELECT over_number + 1 AS over_no, -- what to show
       ROUND(6.0*SUM(total_runs)/SUM(is_legal),2) -- runs per over
       AS runs_per_over
FROM v_ball -- one row = one ball
WHERE over_number IN (0, 19) -- these overs only
GROUP BY over_number;
```

C7 — Is the middle phase really slower than the powerplay?

Source interpretation: The middle overs go at 7.92 and the powerplay at 8.03. The middle is marginally slower, but the gap is tiny.

SQL Query:

```
SELECT phase, -- what to show
       ROUND(6.0*SUM(total_runs)/SUM(is_legal),2) -- runs per over
       AS runs_per_over
FROM v_ball -- one row = one ball
WHERE phase IN ('Powerplay','Middle') -- these phases only
GROUP BY phase;
```

C8 — What share of an innings total comes from each phase?

Source interpretation: The middle overs produce 43.42% of all runs, the powerplay 29.88% and the death 26.71%. The middle matters most simply because it is longest.

SQL Query:

```
SELECT phase, -- what to show
       SUM(total_runs) AS runs,
       ROUND(100.0*SUM(total_runs)/(SELECT -- round it off
SUM(total_runs) FROM
v_ball),2) AS pct_of_all_runs
FROM v_ball -- one row = one ball
GROUP BY phase;
```

C9 — How has powerplay scoring changed across seasons?

Source interpretation: Powerplay scoring was 46.2 in 2008 and 43.3 in 2009. Ask them to run the full nineteen seasons rather than the first four.

SQL Query:

```
SELECT b.match_id, b.innings, -- name a first query
       SUM(b.total_runs) AS pp_runs FROM
v_ball b WHERE b.phase='Powerplay'
GROUP BY b.match_id, b.innings) -- one row per group
SELECT m.season_year, -- what to show
       ROUND(AVG(pp.pp_runs),1) AS -- round it off
       avg_pp_score
FROM pp -- the source table
JOIN matches_clean m ON m.match_id = -- attach the other table
       pp.match_id
GROUP BY m.season_year -- one row per group
ORDER BY m.season_year -- sort
LIMIT 4;
```

Section D — Bowling — the basics

D1 — The team wants the all-time leading wicket-takers.

Source interpretation: YS Chahal leads with 228 wickets. Note that this uses `bowler_wicket`, not `is_wicket` — run outs are not credited to anybody.

SQL Query:

```
SELECT bowler, -- what to show
       SUM(bowler_wicket) AS wickets -- bowler's wickets
FROM v_ball -- one row = one ball
GROUP BY bowler -- one row per group
ORDER BY wickets DESC -- sort
LIMIT 5;
```

D2 — Who is the most economical bowler, counting only runs that were their fault?

Source interpretation: A Kumble is the most economical at 6.58 across 965 balls, with byes and leg-byes correctly removed.

SQL Query:

```
SELECT bowler, -- what to show
       SUM(is_legal) AS balls, -- sample size
       ROUND(6.0*SUM(bowler_runs)/SUM(is_legal),2) -- runs per over conc
       AS economy
FROM v_ball -- one row = one ball
GROUP BY bowler -- one row per group
HAVING balls >= 500 -- filter the groups
ORDER BY economy ASC -- sort
LIMIT 5;
```

D3 — Show how the economy leaderboard changes if byes and leg-byes are charged to the bowler.

Source interpretation: Charging byes and leg-byes to the bowler moves Kumble from 6.58 to 6.77 and reshuffles the order below him. A wrong definition still produces a plausible-looking table.

SQL Query:

```
SELECT bowler, -- what to show
       ROUND(6.0*SUM(total_runs)/SUM(is_legal),2) -- runs per over
       AS wrong_economy
FROM v_ball -- one row = one ball
GROUP BY bowler -- one row per group
HAVING SUM(is_legal) >= 500 -- filter the groups
ORDER BY wrong_economy ASC -- sort
LIMIT 5;
```

D4 — Which bowler takes a wicket most often — best strike rate in balls per wicket?

Source interpretation: AD Russell takes a wicket every 14.68 balls — the best on the list, and he is a batting all-rounder. Strike rate and economy measure different things. `IPLCRICKETANALYTICSCCLASSROOMDELIVERYR`
`UNBBOOK` Trainer-facing · companion to the IPL Trainer Manual Page 141

SQL Query:

```
SELECT bowler, -- what to show
       SUM(is_legal) AS balls, -- sample size
       SUM(bowler_wicket) AS wickets, -- bowler's wickets
       ROUND(1.0*SUM(is_legal)/SUM(bowler_wicket),2) -- bowler's wickets
       AS balls_per_wicket
FROM v_ball -- one row = one ba
GROUP BY bowler -- one row per group
HAVING wickets >= 50 -- filter the groups
ORDER BY balls_per_wicket ASC -- sort
LIMIT 5;
```

D5 — Which bowler gives away the fewest boundaries?

Source interpretation: M Muralitharan concedes a boundary on only 11.15% of balls. Boundary rate is the cleanest measure of control there is.

SQL Query:

```
SELECT bowler, -- what to show
       SUM(is_legal) AS balls, -- sample size
       ROUND(100.0*SUM(CASE WHEN batter_runs -- round it off
IN (4,6) THEN 1 ELSE 0
END)/SUM(is_legal),2) AS -- sample size
       boundary_pct
FROM v_ball -- one row = one ball
GROUP BY bowler -- one row per group
HAVING balls >= 1000 -- filter the groups
```

```
ORDER BY boundary_pct ASC -- sort
LIMIT 5;
```

D6 — Which bowler bowls the most dot balls?

Source interpretation: DW Steyn bowls a dot on 46.83% of balls — nearly half. In T20 a dot ball is close to a wicket in value.

SQL Query:

```
SELECT bowler, -- what to show
       SUM(is_legal) AS balls, -- sample size
       ROUND(100.0*SUM(CASE WHEN -- round it off
total_runs=0 AND is_legal=1
THEN 1 ELSE 0 -- branches of the CASE
END)/SUM(is_legal),2) AS -- sample size
       dot_pct
FROM v_ball -- one row = one ball
GROUP BY bowler -- one row per group
HAVING balls >= 1000 -- filter the groups
ORDER BY dot_pct DESC -- sort
LIMIT 5;
```

D7 — Which bowlers leak the most extras?

Source interpretation: B Kumar has conceded 199 runs in wides and no-balls. Extras are free runs and they are entirely avoidable.

SQL Query:

```
SELECT bowler, -- what to show
       SUM(wide_ball_runs + no_ball_runs) AS
       extras_conceded,
       SUM(is_legal) AS balls -- sample size
FROM v_ball -- one row = one ball
GROUP BY bowler -- one row per group
HAVING balls >= 1000 -- filter the groups
ORDER BY extras_conceded DESC -- sort
LIMIT 5;
```

D8 — How do bowlers actually take their wickets?

SQL Query:

```
SELECT wicket_kind, COUNT(*) AS n -- row count
FROM v_ball -- one row = one ball
WHERE is_wicket = 1 -- wicket balls only
GROUP BY wicket_kind -- one row per group
ORDER BY n DESC;
```

Section E — Pace against spin

E1 — Are spinners cheaper than fast bowlers overall?

Explanation: 4. What the important SQL pieces are doing CASE WHEN creates a value using if/then/else logic. LIKE searches text patterns; % means any number of characters. SUM() adds values across rows. ROUND() makes numeric output easier to read.

Source interpretation: Spin goes at 7.76 an over against pace at 8.57, and takes a wicket on 4.37% of balls against 4.93%. Spin is cheaper; pace strikes more often. IPL CRICKET ANALYTICS • STUDENT SQL BOOKLET Page 6

SQL Query:

```
SELECT CASE WHEN bowler_type LIKE -- spin or pace
'%Legbreak%' OR bowler_type LIKE
'%Offbreak%' OR bowler_type LIKE
'%Orthodox%' OR bowler_type LIKE
'%Wrist spin%' OR bowler_type LIKE
'%Googly%' THEN 'Spin' ELSE 'Pace'
END AS style, -- branches of th
SUM(is_legal) AS balls, -- sample size
ROUND(6.0*SUM(bowler_runs)/SUM(is_legal),2) -- runs per over
AS economy,
ROUND(100.0*SUM(bowler_wicket)/SUM(is_legal),2) -- bowler's wicke
AS wicket_pct
FROM v_ball -- one row = one
WHERE bowler_type IS NOT NULL AND -- drop the blanks
TRIM(bowler_type) <> ''
GROUP BY style;
```

E2 — Which bowling style is most economical of all the styles recorded?

Explanation: 4. What the important SQL pieces are doing HAVING filters groups after aggregation. LIMIT restricts the number of rows displayed. SUM() adds values across rows. ROUND() makes numeric output easier to read.

Source interpretation: IPL CRICKET ANALYTICS • STUDENT SQL BOOKLET Page 7 Right arm Offbreak is the most economical style at 7.50 across 30,008 balls. Four of the five cheapest styles are spin.

SQL Query:

```
SELECT bowler_type, -- what to show
SUM(is_legal) AS balls, -- sample size
ROUND(6.0*SUM(bowler_runs)/SUM(is_legal),2) -- runs per over conc
AS economy
FROM v_ball -- one row = one ball
WHERE TRIM(COALESCE(bowler_type,'')) <> '' -- drop the blanks
GROUP BY bowler_type -- one row per group
HAVING balls >= 3000 -- filter the groups
ORDER BY economy ASC -- sort
LIMIT 5;
```

E3 — Should we bowl spin in the powerplay?

Explanation: 4. What the important SQL pieces are doing CASE WHEN creates a value using if/then/else logic. LIKE searches text patterns; % means any number of characters. SUM() adds values across rows. ROUND() makes numeric output easier to read.

Source interpretation: Spin is cheaper than pace in every phase — 7.69 against 7.89 in the powerplay, 7.55 against 8.10 in the middle, and 9.00 against 10.08 at the death. The powerplay gap is the one that answers the question, and it is the smallest of the three.

SQL Query:

```
SELECT phase, -- what to show
CASE WHEN bowler_type LIKE -- spin or pace
'%Legbreak%' OR bowler_type
LIKE '%Offbreak%' OR
bowler_type LIKE '%Orthodox%'
OR bowler_type LIKE '%Googly%'
OR bowler_type LIKE '%Wrist spin%'
THEN 'Spin' ELSE 'Pace' END AS -- or this -- branches of the CA
style,
ROUND(6.0*SUM(bowler_runs)/SUM(is_legal),2) -- runs per over conc
AS economy
FROM v_ball -- one row = one ball
WHERE TRIM(COALESCE(bowler_type,'')) <> '' -- drop the blanks
GROUP BY phase, style -- one row per group
ORDER BY phase, style;
```

E4 — Do left-arm bowlers trouble right-handed batters more?

Explanation: 4. What the important SQL pieces are doing CASE WHEN creates a value using if/then/else logic. LIKE searches text patterns; % means any number of characters. SUM() adds values across rows. ROUND() makes numeric output

easier to read.

Source interpretation: Left-arm bowlers concede 8.49 to right-handers; right-arm bowlers concede 8.33 to left-handers. The differences are small and not in the direction folklore claims.

SQL Query:

```
SELECT CASE WHEN bowler_type LIKE 'Left%' -- spin or pace
THEN 'Left arm' ELSE 'Right arm' END -- branches of th
AS arm,
batsman_type,
ROUND(6.0*SUM(bowler_runs)/SUM(is_legal),2) -- runs per over
AS economy,
ROUND(100.0*SUM(bowler_wicket)/SUM(is_legal),2) -- bowler's wicke
AS wicket_pct
FROM v_ball -- one row = one

IPL CRICKET ANALYTICS • STUDENT SQL BOOKLET
Page 9
WHERE TRIM(COALESCE(bowler_type,'')) <> '' -- drop the blanks
AND batsman_type IS NOT NULL -- and this too
GROUP BY arm, batsman_type;
```

E5 — Which spinner is the best wicket-taker?

Explanation: 4. What the important SQL pieces are doing LIMIT restricts the number of rows displayed. LIKE searches text patterns; % means any number of characters. SUM() adds values across rows.

Source interpretation: YS Chahal leads the spinners with 228 — which is also the overall leader. The best wicket-taker in IPL history is a leg-spinner. I P L CRICKE T ANALYTICS C L ASSROOM DELIVERY RUNBOOK

SQL Query:

```
SELECT bowler, -- what to show
SUM(bowler_wicket) AS wickets, -- bowler's wickets
SUM(is_legal) AS balls -- sample size
FROM v_ball -- one row = one ball
WHERE bowler_type LIKE '%Legbreak%' OR -- matching styles only
bowler_type LIKE '%Offbreak%' OR
bowler_type LIKE '%Orthodox%' OR
bowler_type LIKE '%Googly%' OR

IPL CRICKET ANALYTICS • STUDENT SQL BOOKLET
Page 10
bowler_type LIKE '%Wrist spin%'
GROUP BY bowler -- one row per group
ORDER BY wickets DESC -- sort
LIMIT 5;
```

E6 — Which fast bowler is the best wicket-taker at the death?

Explanation: 4. What the important SQL pieces are doing HAVING filters groups after aggregation. LIMIT restricts the number of rows displayed. LIKE searches text patterns; % means any number of characters. SUM() adds values across rows.

Source interpretation: B Kumar and DJ Bravo both have 115 death-over wickets. The death is where pace bowlers earn their reputation.

SQL Query:

```
SELECT bowler, -- what to show
SUM(bowler_wicket) AS wickets, -- bowler's wickets
SUM(is_legal) AS balls -- sample size

IPL CRICKET ANALYTICS • STUDENT SQL BOOKLET
Page 11
FROM v_ball -- one row = one ball
WHERE phase='Death' AND (bowler_type LIKE -- death overs only
'%Fast%' OR bowler_type LIKE
'%Medium%')
GROUP BY bowler
HAVING balls >= 200
ORDER BY wickets DESC
LIMIT 5;
```

E7 — Is spin more effective in the middle overs than pace?

Explanation: 4. What the important SQL pieces are doing CASE WHEN creates a value using if/then/else logic. LIKE searches text patterns; % means any number of characters. SUM() adds values across rows. ROUND() makes numeric output easier to read.

Source interpretation: In the middle overs spin goes at 7.55 against pace at 8.10, and both take a wicket about every 24 balls. Spin is cheaper for the same strike rate.

SQL Query:

```

SELECT CASE WHEN bowler_type LIKE
'%Legbreak%' OR bowler_type LIKE
'%Offbreak%' OR bowler_type LIKE
'%Orthodox%' OR bowler_type LIKE
'%Googly%' OR bowler_type LIKE
'%Wrist spin%' THEN 'Spin' ELSE 'Pace'
END AS style,
SUM(is_legal) AS balls,
ROUND(6.0*SUM(bowler_runs)/SUM(is_legal),2)
AS economy,
ROUND(1.0*SUM(is_legal)/NULLIF(SUM(bowler_wicket),0),1)
AS balls_per_wicket
FROM v_ball
WHERE phase='Middle' AND
TRIM(COALESCE(bowler_type,'')) <> ''
GROUP BY style;

```

E8 — How many different bowling styles does the data record?

Explanation: 4. What the important SQL pieces are doing COUNT() counts rows or distinct values, depending on how it is used.

Source interpretation: Twenty distinct bowling styles are recorded, after the Step 2 cleaning merged the capitalisation variant.

SQL Query:

```

SELECT COUNT(DISTINCT bowler_type) AS styles -- distinct values
FROM v_ball -- one row = one ball
WHERE TRIM(COALESCE(bowler_type,'')) <> '';

```

Section F — Bowling specialists — powerplay and death

F1 — We need a death-overs bowler. Who is cheapest in overs 16 to 20?

Explanation: 4. What the important SQL pieces are doing HAVING filters groups after aggregation. LIMIT restricts the number of rows displayed. SUM() adds values across rows. ROUND() makes numeric output easier to read.

Source interpretation: SP Narine goes at 7.29 in the death overs, when the average is 9.86. That is more than two and a half runs an over below the field.

SQL Query:

```
SELECT bowler, -- what to show
       SUM(is_legal) AS balls, -- sample size
       ROUND(6.0*SUM(bowler_runs)/SUM(is_legal),2) -- runs per over conc
       AS death_economy
FROM v_ball -- one row = one ball
WHERE phase='Death' -- death overs only
GROUP BY bowler -- one row per group
HAVING balls >= 300 -- filter the groups
ORDER BY death_economy ASC -- sort
LIMIT 5;
```

F2 — We need a powerplay strike bowler. Who takes most wickets in overs

Explanation: 4. What the important SQL pieces are doing HAVING filters groups after aggregation. LIMIT restricts the number of rows displayed. SUM() adds values across rows. ROUND() makes numeric output easier to read.

Source interpretation: B Kumar has 89 powerplay wickets at an economy of 6.55. Wickets and control at the same time is rare and it is why he is the answer here. IPL CRICKET ANALYTICS • STUDENT SQL BOOKLET Page 16

SQL Query:

```
SELECT bowler, -- what to show
       SUM(bowler_wicket) AS wickets, -- bowler's wickets
       SUM(is_legal) AS balls, -- sample size
       ROUND(6.0*SUM(bowler_runs)/SUM(is_legal),2) -- runs per over conc
       AS economy
FROM v_ball -- one row = one ball
WHERE phase='Powerplay' -- powerplay only
GROUP BY bowler -- one row per group
HAVING balls >= 300 -- filter the groups
ORDER BY wickets DESC -- sort
LIMIT 5;
```

F3 — Which bowler is good in the powerplay but expensive at the death?

Explanation: 4. What the important SQL pieces are doing HAVING filters groups after aggregation. LIMIT restricts the number of rows displayed. IPL CRICKET ANALYTICS • STUDENT SQL BOOKLET Page 17 CASE WHEN creates a value using if/then/else logic. SUM() adds values across rows. ROUND() makes numeric output easier to read.

Source interpretation: P Kumar goes at 6.69 in the powerplay and 10.46 at the death — a gap of nearly four runs an over. He is a specialist, whether or not anybody labelled him one.

SQL Query:

```
SELECT bowler, -- what to show
       ROUND(6.0*SUM(CASE WHEN -- round it off
           phase='Powerplay' THEN
           bowler_runs ELSE 0 END)
       /NULLIF(SUM(CASE WHEN -- divide, no zero risk
           phase='Powerplay' THEN
           is_legal ELSE 0 END),0),2) AS
       econ_pp,
       ROUND(6.0*SUM(CASE WHEN phase='Death' -- round it off
           THEN bowler_runs ELSE 0 END) -- branches of the CASE
       /NULLIF(SUM(CASE WHEN -- divide, no zero risk
           phase='Death' THEN is_legal
           ELSE 0 END),0),2) AS -- branches of the CASE
       econ_death
FROM v_ball -- one row = one ball
GROUP BY bowler -- one row per group
HAVING SUM(CASE WHEN phase='Powerplay' THEN -- filter the groups
           is_legal ELSE 0 END) >= 300 AND
       SUM(CASE WHEN phase='Death' THEN -- if-then-else per row
           is_legal ELSE 0 END) >= 300
ORDER BY (econ_death - econ_pp) DESC -- sort
LIMIT 5;
```

F4 — Who should bowl the 20th over?

Explanation: 4. What the important SQL pieces are doing HAVING filters groups after aggregation. LIMIT restricts the number of rows displayed. SUM() adds values across rows. ROUND() makes numeric output easier to read.

Source interpretation: CH Morris goes at 8.35 in the 20th over, when the over averages 11.12 on the same measure — see F7. C4's 11.39 counts every run the batting side scored; this counts only the runs charged to the bowler. The single most pressured over in cricket, and some bowlers are three runs better at it.

SQL Query:

```
SELECT bowler, -- what to show
SUM(is_legal) AS balls, -- sample size
ROUND(6.0*SUM(bowler_runs)/SUM(is_legal),2) -- runs per over conc
AS economy_last_over
FROM v_ball
WHERE over_number = 19
GROUP BY bowler
HAVING balls >= 100
LIMIT 5;
```

F5 — How much more expensive is the death than the powerplay, for the

Explanation: 4. What the important SQL pieces are doing SUM() adds values across rows. ROUND() makes numeric output easier to read.

Source interpretation: The death costs 9.86 an over against 7.86 in the powerplay — exactly two runs more, for the same bowlers and the same game.

SQL Query:

```
SELECT phase, -- what to show

IPL CRICKET ANALYTICS • STUDENT SQL BOOKLET
Page 19
ROUND(6.0*SUM(bowler_runs)/SUM(is_legal),2) -- runs per over conc
AS economy
FROM v_ball
WHERE phase IN ('Powerplay','Death')
GROUP BY phase;
```

F6 — Which bowler concedes the fewest sixes at the death?

Explanation: 4. What the important SQL pieces are doing HAVING filters groups after aggregation. LIMIT restricts the number of rows displayed. CASE WHEN creates a value using if/then/else logic. SUM() adds values across rows. ROUND() makes numeric output easier to read.

Source interpretation: SL Malinga concedes a six on only 4.48% of death balls. Against the yorker, the six simply does not come.

SQL Query:

```
SELECT bowler, -- what to show
SUM(is_legal) AS balls, -- sample size
ROUND(100.0*SUM(CASE WHEN -- round it off
batter_runs=6 THEN 1 ELSE 0

IPL CRICKET ANALYTICS • STUDENT SQL BOOKLET
Page 20
END)/SUM(is_legal),2) AS -- sample size
six_pct
FROM v_ball -- one row = one ball
WHERE phase='Death' -- death overs only
GROUP BY bowler -- one row per group
HAVING balls >= 300 -- filter the groups
ORDER BY six_pct ASC -- sort
LIMIT 5;
```

F7 — Is it worth saving your best bowler for the death?

Explanation: 4. What the important SQL pieces are doing SUM() adds values across rows. ROUND() makes numeric output easier to read.

Source interpretation: Economy climbs every single over from 8.91 in the 16th to 11.12 in the 20th. There is no plateau; it rises to the last ball.

SQL Query:

```
SELECT over_number + 1 AS over_no, -- what to show
ROUND(6.0*SUM(bowler_runs)/SUM(is_legal),2) -- runs per over conc

IPL CRICKET ANALYTICS • STUDENT SQL BOOKLET
Page 21
AS economy
FROM v_ball -- one row = one ball
WHERE over_number >= 15 -- last five overs on
GROUP BY over_number -- one row per group
ORDER BY over_number;
```

F8 — Which bowler has the best dot-ball rate at the death?

Explanation: 4. What the important SQL pieces are doing HAVING filters groups after aggregation. LIMIT restricts the number of rows displayed. CASE WHEN creates a value using if/then/else logic. SUM() adds values across rows. ROUND() makes numeric output easier to read.

Source interpretation: SP Narine bowls a dot on 36.93% of death balls, when the league average at the death is 28.45%. He appears in F1 as well.

SQL Query:

```
SELECT bowler, -- what to show
SUM(is_legal) AS balls, -- sample size
ROUND(100.0*SUM(CASE WHEN -- round it off
total_runs=0 AND is_legal=1
THEN 1 ELSE 0 -- branches of the CASE

IPL CRICKET ANALYTICS • STUDENT SQL BOOKLET
Page 22
END)/SUM(is_legal),2) AS -- sample size
dot_pct
FROM v_ball
WHERE phase='Death'
GROUP BY bowler
HAVING balls >= 300 -- one row = one ball -- death overs only -- one row
per group -- filter the groups
ORDER BY dot_pct DESC -- sort
LIMIT 5;
```

Section G — The toss

G1 — Does the toss winner win the match more often than not?

Explanation: 4. What the important SQL pieces are doing CASE WHEN creates a value using if/then/else logic. SUM() adds values across rows. COUNT() counts rows or distinct values, depending on how it is used. ROUND() makes numeric output easier to read.

Source interpretation: The toss winner wins 613 of 1,187 decisive matches — 51.64%. Almost exactly a coin flip.

SQL Query:

```
SELECT COUNT(*) AS decisive_matches, -- row count
SUM(CASE WHEN toss_winner =
match_winner THEN 1 ELSE 0
END) AS toss_winner_won,
FROM matches_clean -- branches of the CASE
ROUND(100.0*SUM(CASE WHEN toss_winner -- round it off
= match_winner THEN 1 ELSE 0
END)/COUNT(*),2) AS pct -- row count -- one row = one match
WHERE result = 'win';
```

G2 — What do captains actually choose when they win the toss?

Explanation: 4. What the important SQL pieces are doing COUNT() counts rows or distinct values, depending on how it is used. ROUND() makes numeric output easier to read.

Source interpretation: Captains choose to field 800 times out of 1,212 — two thirds of the time. There is a strong shared belief here, whether or not it is justified.

SQL Query:

```
SELECT toss_decision, -- what to show
COUNT(*) AS n, -- row count
ROUND(100.0*COUNT(*)/(SELECT COUNT(*) -- row count
FROM matches_clean),1) AS pct -- one row = one match
FROM matches_clean -- one row = one match
GROUP BY toss_decision;
```

G3 — Does the toss decision change the result?

Explanation: 4. What the important SQL pieces are doing CASE WHEN creates a value using if/then/else logic. SUM() adds values across rows. COUNT() counts rows or distinct values, depending on how it is used. ROUND() makes numeric output easier to read.

Source interpretation: Toss winners who chose to field won 54.65%. Those who chose to bat won 45.77%. Nearly nine points apart.

SQL Query:

```
SELECT toss_decision, -- what to show
COUNT(*) AS matches, -- row count
SUM(CASE WHEN toss_winner =
match_winner THEN 1 ELSE 0
END) AS toss_winner_won, -- branches of the CASE
ROUND(100.0*SUM(CASE WHEN toss_winner -- round it off
= match_winner THEN 1 ELSE 0
END)/COUNT(*),2) AS pct -- row count
FROM matches_clean -- one row = one match
WHERE result='win' -- decisive matches only
GROUP BY toss_decision;
```

G4 — Has the fielding-first preference grown over the seasons?

Explanation: 4. What the important SQL pieces are doing LIMIT restricts the number of rows displayed. CASE WHEN creates a value using if/then/else logic. SUM() adds values across rows. COUNT() counts rows or distinct values, depending on how it is used. ROUND() makes numeric output easier to read.

Source interpretation: In 2008, 55.2% chose to field. In 2010 it was 35.0%, in 2011 it was 65.8%. The preference is not stable across seasons.

SQL Query:

```
SELECT season_year, -- what to show
ROUND(100.0*SUM(CASE WHEN -- round it off
toss_decision='field' THEN 1
ELSE 0 END)/COUNT(*),1) AS -- row count
pct_field
FROM matches_clean -- one row = one match
GROUP BY season_year -- one row per group
ORDER BY season_year -- sort
LIMIT 4;
```

G5 — Which grounds favour the toss winner the most?

Explanation: 4. What the important SQL pieces are doing HAVING filters groups after aggregation. LIMIT restricts the number of rows displayed. CASE WHEN creates a value using if/then/else logic. SUM() adds values across rows. COUNT() counts rows or distinct values, depending on how it is used. ROUND() makes numeric output easier to read.

Source interpretation: Sawai Mansingh Stadium favours the toss winner most, at 56.1% over 66 matches. Even the strongest ground is only six points from a coin flip.

SQL Query:

```
SELECT venue_clean, -- what to show
COUNT(*) AS matches, -- row count
ROUND(100.0*SUM(CASE WHEN toss_winner -- round it off
= match_winner THEN 1 ELSE 0
END)/COUNT(*),1) AS -- row count
toss_win_pct
FROM matches_clean -- one row = one match
WHERE result='win' -- decisive matches only
GROUP BY venue_clean -- one row per group
HAVING matches >= 30 -- filter the groups
ORDER BY toss_win_pct DESC -- sort
LIMIT 5;
```

G6 — Which team wins the toss most often? (a trick question)

Explanation: 4. What the important SQL pieces are doing LIMIT restricts the number of rows displayed. COUNT() counts rows or distinct values, depending on how it is used.

Source interpretation: Mumbai Indians have won 156 tosses — the most. This is a trick question: the answer is meaningless without knowing how many matches each team played.

SQL Query:

```
SELECT toss_winner, -- what to show
COUNT(*) AS tosses_won -- row count
FROM matches_clean -- one row = one match
GROUP BY toss_winner -- one row per group
ORDER BY tosses_won DESC -- sort
LIMIT 5;
```

G7 — Do teams that choose to field win more when chasing small totals?

Explanation: 4. What the important SQL pieces are doing CASE WHEN creates a value using if/then/else logic. AVG() calculates the mean. COUNT() counts rows or distinct values, depending on how it is used. ROUND() makes numeric output easier to read.

Source interpretation: Chasing a target under 160, teams win 76.5% of the time. Chasing 160 or more, they win 40.6%. That gap explains the fielding preference in G2.

SQL Query:

```
SELECT CASE WHEN first_innings_runs < 160 -- band the totals
THEN 'Under 160' ELSE '160 or more' -- branches of the CASE
END AS target_band, -- branches of the CASE
COUNT(*) AS matches,
ROUND(100.0*AVG(chase_won),1) AS
chase_win_pct
FROM v_match_totals -- row count -- chase win rate -- one row = one match
GROUP BY target_band;
```

G8 — Is winning the toss worth anything at all, in one number?

Explanation: 4. What the important SQL pieces are doing CASE WHEN creates a value using if/then/else logic. SUM() adds values across rows. COUNT() counts rows or distinct values, depending on how it is used. ROUND() makes numeric output easier to read.

Source interpretation: Winning the toss is worth 1.64 percentage points over a coin flip. That is the entire effect, stated as one number.

SQL Query:

```
SELECT ROUND(100.0*SUM(CASE WHEN -- what to show
toss_winner=match_winner THEN 1 ELSE

IPL CRICKET ANALYTICS • STUDENT SQL BOOKLET
Page 30
0 END)/COUNT(*),2) - 50.0 AS -- row count
advantage_over_coin_flip
FROM matches_clean
WHERE result='win';
```

Section H — Batting first against chasing

H1 — Overall, is it better to bat first or to chase?

Explanation: 4. What the important SQL pieces are doing SUM() adds values across rows. AVG() calculates the mean. COUNT() counts rows or distinct values, depending on how it is used. ROUND() makes numeric output easier to read.

Source interpretation: The chasing team wins 647 of 1,187 — 54.51%. Chasing carries a real advantage, worth about four and a half points.

SQL Query:

```
SELECT COUNT(*) AS matches, -- row count
       SUM(chase_won) AS chases_won,
       ROUND(100.0*AVG(chase_won),2) AS -- chase win rate
       chase_win_pct
FROM v_match_totals;
```

H2 — What first-innings total is worth defending?

Explanation: 4. What the important SQL pieces are doing CASE WHEN creates a value using if/then/else logic. AVG() calculates the mean. COUNT() counts rows or distinct values, depending on how it is used. ROUND() makes numeric output easier to read.

Source interpretation: The chase win rate falls from 82.6% under 140 to 23.2% at 200 or more. This is the strongest and cleanest finding in the entire project.

SQL Query:

```
SELECT CASE WHEN first_innings_runs < 140 -- band the totals
THEN 'Under 140' WHEN -- branches of the CASE
first_innings_runs < 160 THEN
'140-159' WHEN first_innings_runs <
180 THEN '160-179' WHEN
first_innings_runs < 200 THEN
'180-199' ELSE '200+' END AS
target_band,
COUNT(*) AS matches, -- row count
ROUND(100.0*AVG(chase_won),1) AS -- chase win rate
chase_win_pct
FROM v_match_totals -- one row = one match
GROUP BY target_band -- one row per group
ORDER BY MIN(first_innings_runs);
```

H3 — What is the average first-innings score?

Explanation: 4. What the important SQL pieces are doing AVG() calculates the mean. ROUND() makes numeric output easier to read.

Source interpretation: The average first-innings score is 168.2. That is the number every other total in this area should be compared against.

SQL Query:

```
SELECT ROUND(AVG(first_innings_runs),1) AS -- what to show
avg_first_innings
FROM v_match_totals;
```

H4 — Where is the break-even total — the score at which the game is

Explanation: 4. What the important SQL pieces are doing CASE WHEN creates a value using if/then/else logic. AVG() calculates the mean. COUNT() counts rows or distinct values, depending on how it is used. ROUND() makes numeric output easier to read.

Source interpretation: The crossover sits between 165 and 174, where the chase win rate passes through 48.3%. Below that band the chase is favoured; above it the defence is.

SQL Query:

```
SELECT CASE WHEN first_innings_runs < 155 -- band the totals
THEN 'Under 155' WHEN -- branches of the CASE
first_innings_runs < 165 THEN
'155-164' WHEN first_innings_runs <
175 THEN '165-174' ELSE '175+' END AS
band,
COUNT(*) AS matches,
ROUND(100.0*AVG(chase_won),1) AS
chase_win_pct
FROM v_match_totals
GROUP BY band -- row count -- chase win rate -- one row = one
match -- one row per group
ORDER BY MIN(first_innings_runs);
```

H5 — Do chasing teams score faster than teams batting first?

Explanation: 4. What the important SQL pieces are doing SUM() adds values across rows. ROUND() makes numeric output easier to read.

Source interpretation: First innings scores at 8.49 an over, second innings at 8.37. Chasing teams score marginally slower, not faster.

SQL Query:

```
SELECT innings, -- what to show
       ROUND(6.0*SUM(total_runs)/SUM(is_legal),2) -- runs per over
       AS run_rate
FROM v_ball -- one row = one ball
WHERE innings IN (1,2) -- filter the rows
GROUP BY innings;
```

H6 — Do teams lose more wickets batting first or second?

Explanation: 4. What the important SQL pieces are doing AVG() calculates the mean. ROUND() makes numeric output easier to read.

Source interpretation: Teams batting first lose 6.13 wickets on average; chasing teams lose 5.70. Chasing teams also finish more innings early, having won.

SQL Query:

```
SELECT innings, -- what to show
       ROUND(AVG(wickets),2) AS -- round it off
       avg_wickets_lost
FROM v_innings -- one row = one innings
WHERE innings IN (1,2) -- filter the rows
GROUP BY innings;
```

H7 — Has the advantage of chasing changed over the seasons?

Explanation: 4. What the important SQL pieces are doing LIMIT restricts the number of rows displayed. AVG() calculates the mean. COUNT() counts rows or distinct values, depending on how it is used. ROUND() makes numeric output easier to read.

Source interpretation: In 2026 chasing teams won 63.4%; in 2023 only 45.2%. The advantage moves around a lot season to season.

SQL Query:

```
SELECT season_year, -- what to show
       COUNT(*) AS matches, -- row count
       ROUND(100.0*AVG(chase_won),1) AS -- chase win rate
       chase_win_pct
FROM v_match_totals -- one row = one match
GROUP BY season_year -- one row per group
ORDER BY season_year DESC -- sort

IPL CRICKET ANALYTICS • STUDENT SQL BOOKLET
Page 37
LIMIT 5;
```

H8 — How often is a 200+ total successfully chased?

Explanation: 4. What the important SQL pieces are doing SUM() adds values across rows. COUNT() counts rows or distinct values, depending on how it is used.

Source interpretation: Of 198 matches where a team scored 200 or more, 46 were still chased down. Even a huge total is not safe.

SQL Query:

```
SELECT COUNT(*) AS matches_with_200_plus, -- row count
       SUM(chase_won) AS successfully_chased
FROM v_match_totals -- one row = one match
WHERE first_innings_runs >= 200;
```

H9 — What is the biggest total ever defended, and the biggest ever

Explanation: 4. What the important SQL pieces are doing CASE WHEN creates a value using if/then/else logic.

Source interpretation: The biggest total successfully defended is 287. The biggest successfully chased is 264. Both are extraordinary and both are real. IPL CRICKET ANALYTICS • STUDENT SQL BOOKLET Page 39

SQL Query:

```
SELECT MAX(CASE WHEN chase_won = 0 THEN -- if-then-else per row
first_innings_runs END) AS
biggest_defended,
```

```
MAX(CASE WHEN chase_won = 1 THEN -- if-then-else per row
first_innings_runs END) AS
biggest_chased
FROM v_match_totals;
```

Section I — Venues and conditions

I1 — Which ground has hosted the most matches?

Explanation: 4. What the important SQL pieces are doing LIMIT restricts the number of rows displayed. COUNT() counts rows or distinct values, depending on how it is used.

Source interpretation: Wankhede Stadium leads with 130 matches. Before the Step 2 cleaning this query answered Eden Gardens with 77.

SQL Query:

```
SELECT venue_clean, COUNT(*) AS matches -- row count
FROM matches_clean -- one row = one match
GROUP BY venue_clean -- one row per group
ORDER BY matches DESC -- sort
LIMIT 5;
```

I2 — Which ground is the highest scoring?

Explanation: 4. What the important SQL pieces are doing HAVING filters groups after aggregation. LIMIT restricts the number of rows displayed. AVG() calculates the mean. COUNT() counts rows or distinct values, depending on how it is used. ROUND() makes numeric output easier to read. JOIN connects related rows from two sources using a matching key.

Source interpretation: Arun Jaitley Stadium averages 180.0 an innings, the highest of the well-used grounds.

SQL Query:

```
SELECT m.venue_clean, -- what to show
COUNT(DISTINCT m.match_id) AS -- distinct values
matches,
ROUND(AVG(i.runs),1) AS -- round it off
avg_innings_score
FROM matches_clean m -- one row = one match
JOIN v_innings i ON i.match_id = -- attach the other table
m.match_id
GROUP BY m.venue_clean -- one row per group
HAVING matches >= 25 -- filter the groups
ORDER BY avg_innings_score DESC -- sort
LIMIT 5;
```

I3 — Which ground favours the chasing team?

Explanation: 4. What the important SQL pieces are doing HAVING filters groups after aggregation. LIMIT restricts the number of rows displayed. AVG() calculates the mean. COUNT() counts rows or distinct values, depending on how it is used. ROUND() makes numeric output easier to read.

Source interpretation: Sawai Mansingh Stadium has a 65.2% chase win rate over 66 matches — more than ten points above the league average.

SQL Query:

```
SELECT venue_clean, -- what to show
COUNT(*) AS matches, -- row count
ROUND(100.0*AVG(chase_won),1) AS -- chase win rate
chase_win_pct
FROM v_match_totals -- one row = one match
GROUP BY venue_clean -- one row per group
HAVING matches >= 30 -- filter the groups
ORDER BY chase_win_pct DESC -- sort
LIMIT 5;
```

I4 — Which ground helps the spinners most?

Explanation: 4. What the important SQL pieces are doing HAVING filters groups after aggregation. LIMIT restricts the number of rows displayed. CASE WHEN creates a value using if/then/else logic. LIKE searches text patterns; % means any number of characters. SUM() adds values across rows. IPL CRICKET ANALYTICS • STUDENT SQL BOOKLET Page 44 COUNT() counts rows or distinct values, depending on how it is used. ROUND() makes numeric output easier to read.

Source interpretation: Spinners go at 7.24 an over at MA Chidambaram Stadium across 95 matches — the friendliest ground for spin.

SQL Query:

```
SELECT m.venue_clean, -- what to show
ROUND(6.0*SUM(CASE WHEN b.bowler_type -- round it off
LIKE '%Legbreak%' OR
b.bowler_type LIKE
'%Offbreak%' OR b.bowler_type
LIKE '%Orthodox%' OR
b.bowler_type LIKE '%Googly%'
OR b.bowler_type LIKE '%Wrist spin%'
```

```

THEN b.bowler_runs ELSE 0 END) -- branches of the CASE
/NULLIF(SUM(CASE WHEN -- divide, no zero risk
b.bowler_type LIKE
'%Legbreak%' OR b.bowler_type
LIKE '%Offbreak%' OR
b.bowler_type LIKE
'%Orthodox%' OR b.bowler_type
LIKE '%Googly%' OR b.bowler_type
LIKE '%Wrist spin%' THEN
b.is_legal ELSE 0 END),0),2)
IPL CRICKET ANALYTICS CLASSROOM DELIVERY RUNBOOK
AS spin_economy,
COUNT(DISTINCT m.match_id) AS matches -- distinct values
FROM v_ball b -- one row = one ball
JOIN matches_clean m ON m.match_id = -- attach the other table
b.match_id
GROUP BY m.venue_clean -- one row per group
HAVING matches >= 25 -- filter the groups
ORDER BY spin_economy ASC -- sort
LIMIT 5;

```

15 — How many separate grounds has the IPL been played at?

Explanation: 4. What the important SQL pieces are doing COUNT() counts rows or distinct values, depending on how it is used.

Source interpretation: 41 separate grounds have hosted IPL cricket, after the venue names were merged in Step 2.

SQL Query:

```

SELECT COUNT(DISTINCT venue_clean) AS -- distinct values
grounds
FROM matches_clean;

```

16 — Which ground has the lowest average score?

Explanation: 4. What the important SQL pieces are doing HAVING filters groups after aggregation. LIMIT restricts the number of rows displayed. AVG() calculates the mean. COUNT() counts rows or distinct values, depending on how it is used. ROUND() makes numeric output easier to read. JOIN connects related rows from two sources using a matching key.

Source interpretation: Sheikh Zayed Stadium averages 152.2, nearly 28 runs below Arun Jaitley. The same game, two very different scoreboards.

SQL Query:

```

SELECT m.venue_clean, -- what to show
COUNT(DISTINCT m.match_id) AS -- distinct values
matches,
ROUND(AVG(i.runs),1) AS avg_score -- round it off
FROM matches_clean m -- one row = one match
JOIN v_innings i ON i.match_id = -- attach the other table
m.match_id
GROUP BY m.venue_clean -- one row per group
HAVING matches >= 25 -- filter the groups
ORDER BY avg_score ASC -- sort
LIMIT 5;

```

17 — Does the powerplay behave differently at different grounds?

Explanation: 4. What the important SQL pieces are doing HAVING filters groups after aggregation. LIMIT restricts the number of rows displayed. SUM() adds values across rows. COUNT() counts rows or distinct values, depending on how it is used. ROUND() makes numeric output easier to read. JOIN connects related rows from two sources using a matching key.

Source interpretation: IPL CRICKET ANALYTICS • STUDENT SQL BOOKLET Page 47 The powerplay goes at 9.41 an over at Arun Jaitley and lower everywhere else. Ground conditions show up in the first six overs.

SQL Query:

```

SELECT m.venue_clean, -- what to show
ROUND(6.0*SUM(b.total_runs)/SUM(b.is_legal),2) -- round it off
AS pp_run_rate,
COUNT(DISTINCT m.match_id) AS matches -- distinct values
FROM v_ball b -- one row = one b
JOIN matches_clean m ON -- attach the othe
m.match_id=b.match_id
WHERE b.phase='Powerplay' -- powerplay only
GROUP BY m.venue_clean -- one row per gro
HAVING matches >= 25 -- filter the grou
ORDER BY pp_run_rate DESC -- sort
LIMIT 5;

```

18 — Can we compare day matches with night matches?

Explanation: 4. What the important SQL pieces are doing COUNT() counts rows or distinct values, depending on how it is used.

Source interpretation: 1,212 matches, and all 1,212 have a date. But the date has no time of day in it, so day and night matches cannot be separated.

SQL Query:

```
SELECT COUNT(*) AS matches, -- row count
       COUNT(match_date) AS have_a_date
FROM matches_clean;
```

Section J — Teams

J3 — Which team scores fastest in the powerplay?

Explanation: 4. What the important SQL pieces are doing HAVING filters groups after aggregation. LIMIT restricts the number of rows displayed. SUM() adds values across rows. ROUND() makes numeric output easier to read.

Source interpretation: Punjab Kings score fastest in the powerplay at 8.21 an over, just ahead of Sunrisers at 8.20.

SQL Query:

```
SELECT batting_team, -- what to show
       ROUND(6.0*SUM(total_runs)/SUM(is_legal),2) -- runs per over
       AS pp_run_rate
FROM v_ball -- one row = one ball
WHERE phase='Powerplay' -- powerplay only
GROUP BY batting_team -- one row per group
HAVING SUM(is_legal) >= 3000 -- filter the groups
ORDER BY pp_run_rate DESC -- sort
LIMIT 5;
```

J4 — Which team has the best death-overs bowling?

Explanation: 4. What the important SQL pieces are doing HAVING filters groups after aggregation. LIMIT restricts the number of rows displayed. SUM() adds values across rows. ROUND() makes numeric output easier to read.

Source interpretation: Chennai Super Kings have the best death-overs economy at 9.53, in a league averaging 9.86.

SQL Query:

```
SELECT bowling_team, -- what to show
       ROUND(6.0*SUM(bowler_runs)/SUM(is_legal),2) -- runs per over conc
       AS death_economy
FROM v_ball
WHERE phase='Death'
GROUP BY bowling_team
HAVING SUM(is_legal) >= 2000
ORDER BY death_economy ASC
LIMIT 5;
```

J6 — Which team is best at defending a total?

Explanation: 4. What the important SQL pieces are doing HAVING filters groups after aggregation. LIMIT restricts the number of rows displayed. AVG() calculates the mean. COUNT() counts rows or distinct values, depending on how it is used. ROUND() makes numeric output easier to read.

Source interpretation: Mumbai Indians successfully defend 53.1% of the time from 145 innings batting first, the best on the list.

SQL Query:

```
SELECT bat_first_team, -- what to show
       COUNT(*) AS defended_innings, -- row count
       ROUND(100.0*AVG(1 - chase_won),1) AS -- round it off
       defence_win_pct
FROM v_match_totals -- one row = one match
GROUP BY bat_first_team -- one row per group
HAVING defended_innings >= 50 -- filter the groups
ORDER BY defence_win_pct DESC -- sort
LIMIT 5;
```

J7 — Which team relies most on boundaries for its runs?

Explanation: 4. What the important SQL pieces are doing HAVING filters groups after aggregation. LIMIT restricts the number of rows displayed. CASE WHEN creates a value using if/then/else logic. SUM() adds values across rows. ROUND() makes numeric output easier to read.

Source interpretation: Mumbai Indians take 62.30% of their runs in boundaries — the most boundary-dependent team.

SQL Query:

```
SELECT batting_team, -- what to show
       ROUND(100.0*SUM(CASE WHEN batter_runs -- round it off
IN (4,6) THEN batter_runs ELSE
0 END) /SUM(batter_runs),2) AS -- total runs
       pct_from_boundaries
FROM v_ball -- one row = one ball
GROUP BY batting_team -- one row per group
HAVING SUM(is_legal) >= 10000 -- filter the groups
ORDER BY pct_from_boundaries DESC -- sort
LIMIT 5;
```

Section K — Wickets, dismissals and pressure

K1 — How often does a wicket fall, per ball?

Explanation: 4. What the important SQL pieces are doing SUM() adds values across rows. ROUND() makes numeric output easier to read.

Source interpretation: A wicket falls on 5.16% of legal balls — roughly one every 19.4 balls, or a bit more than three an over.

SQL Query:

```
SELECT -- what to show
ROUND(100.0*SUM(is_wicket)/SUM(is_legal),2) -- wicket chance per
AS wicket_pct_per_ball,
ROUND(1.0*SUM(is_legal)/SUM(is_wicket),1)
AS balls_per_wicket
FROM v_ball;
```

K2 — Which fielder has been involved in the most dismissals? (a trap)

Explanation: 4. What the important SQL pieces are doing LIMIT restricts the number of rows displayed. COUNT() counts rows or distinct values, depending on how it is used.

Source interpretation: The most common value is the text '[None]' on 3,892 rows — 16 fewer than the 3,908 recorded in Step 2, because v_ball drops the super-over balls —, and every real answer is buried inside a list in one cell. This question cannot be answered from the raw column.

SQL Query:

```
SELECT fielders_involved, COUNT(*) AS n -- row count
FROM v_ball -- one row = one ball
WHERE is_wicket=1
GROUP BY fielders_involved
ORDER BY n DESC
LIMIT 5;
```

K3 — How does the way batters get out change across the phases?

Explanation: 4. What the important SQL pieces are doing COUNT() counts rows or distinct values, depending on how it is used.

Source interpretation: Caught is the most common dismissal in every phase, but it rises from 3,484 in the middle to 3,289 in the death from far fewer balls.

SQL Query:

```
SELECT phase, -- what to show
wicket_kind,
COUNT(*) AS n -- row count
FROM v_ball -- one row = one ball
WHERE is_wicket=1 AND wicket_kind IN -- that dismissal only
('caught','bowled','run out')
GROUP BY phase, wicket_kind -- one row per group
ORDER BY phase, n DESC;
```

K4 — Which batter is most often run out?

Explanation: 4. What the important SQL pieces are doing LIMIT restricts the number of rows displayed. COUNT() counts rows or distinct values, depending on how it is used.

Source interpretation: S Dhawan and G Gambhir are each run out 16 times. Run outs are the one dismissal nobody is credited with.

SQL Query:

```
SELECT player_out, COUNT(*) AS run_outs -- row count
FROM v_ball -- one row = one ball
WHERE wicket_kind='run out'
GROUP BY player_out
ORDER BY run_outs DESC
LIMIT 5;
```

K5 — Which bowler dismisses a named batter most often?

Explanation: 4. What the important SQL pieces are doing LIMIT restricts the number of rows displayed. COUNT() counts rows or distinct values, depending on how it is used.

Source interpretation: Sandeep Sharma has dismissed V Kohli 7 times, more than any other bowler. Matchup data like this drives real team selection.

SQL Query:

```

SELECT bowler, COUNT(*) AS times_out -- row count
FROM v_ball -- one row = one ball
WHERE player_out='V Kohli' AND -- that batter's outs
bowler_wicket=1
GROUP BY bowler
ORDER BY times_out DESC
LIMIT 5;

```

K6 — How many wickets fall in a typical innings?

Explanation: 4. What the important SQL pieces are doing AVG() calculates the mean. ROUND() makes numeric output easier to read.

Source interpretation: An average innings loses 5.92 wickets. Teams almost never use all ten, which is what makes T20 different from other formats.

SQL Query:

```

SELECT ROUND(AVG(wickets),2) AS -- what to show
avg_wickets_per_innings
FROM v_innings;

```

K7 — Do teams that lose early wickets still post a good score?

Explanation: 4. What the important SQL pieces are doing GROUP BY creates one result row for each group. LIMIT restricts the number of rows displayed. SUM() adds values across rows. AVG() calculates the mean. COUNT() counts rows or distinct values, depending on how it is used. ROUND() makes numeric output easier to read. JOIN connects related rows from two sources using a matching key.

Source interpretation: Losing no powerplay wickets leads to an average of 171.3. Losing four leads to 128.3. Every early wicket costs roughly 15 runs.

SQL Query:

```

SELECT match_id, innings, -- name a first query
SUM(is_wicket) AS pp_wkts FROM v_ball -- wickets
WHERE phase='Powerplay' GROUP BY -- powerplay only
match_id, innings)
SELECT pp.pp_wkts AS -- what to show
wickets_lost_in_powerplay,
COUNT(*) AS innings_count, -- row count
ROUND(AVG(i.runs),1) AS -- round it off
avg_final_score
FROM pp -- the source table
JOIN v_innings i ON i.match_id=pp.match_id -- attach the other table
AND i.innings=pp.innings -- and this too
GROUP BY pp.pp_wkts -- one row per group
ORDER BY pp.pp_wkts -- sort
LIMIT 5;

```

K8 — What share of dismissals are caught?

Explanation: 4. What the important SQL pieces are doing CASE WHEN creates a value using if/then/else logic. SUM() adds values across rows. COUNT() counts rows or distinct values, depending on how it is used. ROUND() makes numeric output easier to read.

Source interpretation: 66.0% of all dismissals are caught, including caught and bowled. Two out of every three wickets involve a catch.

SQL Query:

```

SELECT ROUND(100.0*SUM(CASE WHEN wicket_kind -- what to show
IN ('caught', 'caught and bowled')
THEN 1 ELSE 0 END) /COUNT(*),2) AS
pct_caught
FROM v_ball
WHERE is_wicket=1;

```

Section L — Seasons — how the game has changed

L1 — Has scoring gone up across the seasons?

Explanation: 4. What the important SQL pieces are doing LIMIT restricts the number of rows displayed. AVG() calculates the mean. ROUND() makes numeric output easier to read. JOIN connects related rows from two sources using a matching key.

Source interpretation: The average innings score was 184.0 in 2026 and 164.8 in 2022. Scoring has risen sharply in recent seasons.

SQL Query:

```
SELECT m.season_year, -- what to show
ROUND(AVG(i.runs),1) AS -- round it off
avg_innings_score
FROM matches_clean m -- one row = one match
JOIN v_innings i ON i.match_id=m.match_id -- attach the other table
GROUP BY m.season_year
ORDER BY m.season_year DESC
LIMIT 5;
```

L2 — Has the six become more common?

Explanation: 4. What the important SQL pieces are doing LIMIT restricts the number of rows displayed. CASE WHEN creates a value using if/then/else logic. SUM() adds values across rows. ROUND() makes numeric output easier to read. JOIN connects related rows from two sources using a matching key.

Source interpretation: Sixes accounted for 8.30% of balls in 2026 against 6.20% in 2022. Roughly one extra six every three overs.

SQL Query:

```
SELECT m.season_year, -- what to show
ROUND(100.0*SUM(CASE WHEN -- round it off
b.batter_runs=6 THEN 1 ELSE 0
END)/SUM(b.is_legal),2) AS -- branches of the CASE
six_pct
FROM v_ball b -- one row = one ball
JOIN matches_clean m ON -- attach the other table
m.match_id=b.match_id
GROUP BY m.season_year -- one row per group
ORDER BY m.season_year DESC -- sort
LIMIT 5;
```

L3 — Has the dot-ball rate fallen?

Explanation: 4. What the important SQL pieces are doing LIMIT restricts the number of rows displayed. CASE WHEN creates a value using if/then/else logic. SUM() adds values across rows. ROUND() makes numeric output easier to read. JOIN connects related rows from two sources using a matching key.

Source interpretation: The dot-ball rate has fallen from 37.08% in 2022 to 32.45% in 2026. Batters are finding a way to score off more balls.

SQL Query:

```
SELECT m.season_year, -- what to show
ROUND(100.0*SUM(CASE WHEN -- round it off
b.total_runs=0 AND
b.is_legal=1 THEN 1 ELSE 0
END)/SUM(b.is_legal),2) AS -- branches of the CASE
dot_pct
FROM v_ball b -- one row = one ball
JOIN matches_clean m ON -- attach the other table
m.match_id=b.match_id
GROUP BY m.season_year -- one row per group
ORDER BY m.season_year DESC -- sort
LIMIT 5;
```

L4 — How many matches were played each season?

Explanation: 4. What the important SQL pieces are doing LIMIT restricts the number of rows displayed. COUNT() counts rows or distinct values, depending on how it is used.

Source interpretation: 2013 had the most matches at 76, then several seasons at 74. Season length has varied and that affects every per-season total.

SQL Query:

```
SELECT season_year, COUNT(*) AS matches -- row count
FROM matches_clean -- one row = one match
GROUP BY season_year
ORDER BY matches DESC
```

```
LIMIT 5;
```

L5 — Has the death-overs run rate changed?

Explanation: 4. What the important SQL pieces are doing LIMIT restricts the number of rows displayed. SUM() adds values across rows. ROUND() makes numeric output easier to read. JOIN connects related rows from two sources using a matching key.

Source interpretation: Death-over scoring reached 11.21 an over in 2024. The last five overs have got faster along with everything else. Page 176 Trainer-facing · companion to the IPL Trainer Manual IPL CRICKET ANALYTICS CLASSROOM DELIVERY RUNBOOK Trainer-facing · companion to the IPL Trainer Manual Page 177

SQL Query:

```
SELECT m.season_year, -- what to show
ROUND(6.0*SUM(b.total_runs)/SUM(b.is_legal),2) -- round it off
AS death_run_rate
FROM v_ball b
JOIN matches_clean m ON
m.match_id=b.match_id
WHERE b.phase='Death'
GROUP BY m.season_year
LIMIT 5;
```

L6 — Which season had the highest-scoring matches?

Explanation: 4. What the important SQL pieces are doing GROUP BY creates one result row for each group. LIMIT restricts the number of rows displayed. SUM() adds values across rows. AVG() calculates the mean. ROUND() makes numeric output easier to read. JOIN connects related rows from two sources using a matching key.

Source interpretation: 2024 was the highest-scoring season at 365.8 combined runs a match, just ahead of 2026 on 363.7.

SQL Query:

```
SELECT match_id, -- name a first query
SUM(total_runs) AS runs FROM v_ball
GROUP BY match_id) -- one row per group
SELECT m.season_year, -- what to show
ROUND(AVG(p.runs),1) AS -- round it off
avg_match_runs
FROM per_match p -- the source table
JOIN matches_clean m ON -- attach the other table
m.match_id=p.match_id
GROUP BY m.season_year -- one row per group
ORDER BY avg_match_runs DESC -- sort
LIMIT 5;
```

L7 — Are spinners bowling more or fewer overs than they used to?

Explanation: 4. What the important SQL pieces are doing LIMIT restricts the number of rows displayed. CASE WHEN creates a value using if/then/else logic. LIKE searches text patterns; % means any number of characters. SUM() adds values across rows. ROUND() makes numeric output easier to read. JOIN connects related rows from two sources using a matching key. IPL CRICKET ANALYTICS • STUDENT SQL BOOKLET Page 69

Source interpretation: Spin bowled 38.4% of balls in 2023 and only 26.8% in 2026. Teams are using markedly less spin than they were.

SQL Query:

```
SELECT m.season_year, -- what to show
ROUND(100.0*SUM(CASE WHEN -- round it off
b.bowler_type LIKE
'%Legbreak%' OR b.bowler_type
LIKE '%Offbreak%' OR
b.bowler_type LIKE
'%Orthodox%' OR b.bowler_type
LIKE '%Googly%' OR b.bowler_type
LIKE '%Wrist spin%' THEN
b.is_legal ELSE 0
END)/SUM(b.is_legal),1) AS -- branches of the CASE
pct_balls_spin
FROM v_ball b -- one row = one ball
JOIN matches_clean m ON -- attach the other table
m.match_id=b.match_id
GROUP BY m.season_year -- one row per group
IPL CRICKET ANALYTICS CLASSROOM DELIVERY RUNBOOK
ORDER BY m.season_year DESC -- sort
LIMIT 5;
```

L8 — Season on season, how much did the average score change?

Explanation: 4. What the important SQL pieces are doing GROUP BY creates one result row for each group. ORDER BY controls the final sorting. LIMIT restricts the number of rows displayed. AVG() calculates the mean. ROUND() makes numeric

output easier to read. LAG() looks at a previous row without removing the current row. IPL CRICKET ANALYTICS • STUDENT SQL BOOKLET Page 70 JOIN connects related rows from two sources using a matching key.

Source interpretation: 2024 rose 8.1 runs on the previous season and 2023 rose 9.9. LAG puts the change next to the value that produced it.

SQL Query:

```
SELECT m.season_year AS -- name a first query
       yr, AVG(i.runs) AS avg_runs FROM
       matches_clean m JOIN v_innings i ON
       i.match_id=m.match_id GROUP BY
       m.season_year)
SELECT yr, -- what to show
       ROUND(avg_runs,1) AS avg_runs, -- round it off
       ROUND(avg_runs - LAG(avg_runs) OVER -- previous row's value
       (ORDER BY yr),1) AS
       change_from_last_season
FROM per_season -- the source table
ORDER BY yr DESC -- sort
LIMIT 5;
```

Section M — Scouting — putting it all together

M1 — Find an opener who scores fast AND survives the powerplay.

Explanation: 4. What the important SQL pieces are doing HAVING filters groups after aggregation. LIMIT restricts the number of rows displayed. CASE WHEN creates a value using if/then/else logic. SUM() adds values across rows. ROUND() makes numeric output easier to read.

Source interpretation: Abhishek Sharma strikes at 172.98 in the powerplay from 744 balls, and was dismissed 31 times in that phase. Speed and survival, side by side.

SQL Query:

```
SELECT batter, -- what to show
SUM(is_legal) AS balls, -- sample size
ROUND(100.0*SUM(batter_runs)/SUM(is_legal),2) -- runs per 100 bal
AS sr_pp,
SUM(CASE WHEN player_out = batter
THEN 1 ELSE 0 END) AS -- branches of the
times_out_in_pp
FROM v_ball -- one row = one ba
WHERE phase='Powerplay' -- powerplay only
GROUP BY batter -- one row per group
HAVING balls >= 600 AND sr_pp >= 140 -- filter the groups
ORDER BY sr_pp DESC -- sort
LIMIT 5;
```

M2 — Find a genuine all-rounder: 1000+ runs and 50+ wickets.

Explanation: 4. What the important SQL pieces are doing HAVING filters groups after aggregation. LIMIT restricts the number of rows displayed. SUM() adds values across rows.

Source interpretation: Eight players clear 1,000 runs and 50 wickets. SR Watson leads on runs; RA Jadeja has by far the most wickets of the group.

SQL Query:

```
SELECT b.batter AS player, -- what to show
SUM(b.batter_runs) AS runs,
(SELECT SUM(x.bowler_wicket) FROM
v_ball x WHERE x.bowler =
b.batter) AS wickets
FROM v_ball b -- one row = one ball
GROUP BY b.batter -- one row per group
HAVING runs >= 1000 AND wickets >= 50 -- filter the groups
ORDER BY runs DESC -- sort
LIMIT 8;
```

M3 — Find a bowler who is economical in every phase, not just one.

Explanation: 4. What the important SQL pieces are doing IPL CRICKET ANALYTICS • STUDENT SQL BOOKLET Page 74 HAVING filters groups after aggregation. LIMIT restricts the number of rows displayed. CASE WHEN creates a value using if/then/else logic. SUM() adds values across rows. ROUND() makes numeric output easier to read.

Source interpretation: SP Narine is under 8 in all three phases — 6.86, 6.55 and 7.29. Only a handful of bowlers in nineteen years manage that.

SQL Query:

```
SELECT bowler, -- what to show
ROUND(6.0*SUM(CASE WHEN -- round it off
phase='Powerplay' THEN
bowler_runs ELSE 0 END)
/NULLIF(SUM(CASE WHEN -- divide, no zero risk
phase='Powerplay' THEN
is_legal ELSE 0 END),0),2) AS
pp,
ROUND(6.0*SUM(CASE WHEN -- round it off
phase='Middle' THEN
bowler_runs ELSE 0 END)
/NULLIF(SUM(CASE WHEN -- divide, no zero risk
phase='Middle' THEN is_legal
ELSE 0 END),0),2) AS mid, -- branches of the CASE
ROUND(6.0*SUM(CASE WHEN phase='Death' -- round it off
THEN bowler_runs ELSE 0 END) -- branches of the CASE
/NULLIF(SUM(CASE WHEN -- divide, no zero risk
phase='Death' THEN is_legal
ELSE 0 END),0),2) AS death -- branches of the CASE
FROM v_ball -- one row = one ball
GROUP BY bowler -- one row per group
HAVING SUM(CASE WHEN phase='Powerplay' THEN -- filter the groups
is_legal ELSE 0 END) >= 300 AND
SUM(CASE WHEN phase='Middle' THEN -- if-then-else per row
is_legal ELSE 0 END) >= 300 AND
```

```
SUM(CASE WHEN phase='Death' THEN -- if-then-else per row
is_legal ELSE 0 END) >= 300 AND pp <
8 AND mid < 8 AND death < 9
ORDER BY (pp + mid + death) ASC -- sort
LIMIT 5;
```

M4 — Our budget allows one overseas batter. Who performs best when

Explanation: 4. What the important SQL pieces are doing HAVING filters groups after aggregation. LIMIT restricts the number of rows displayed. SUM() adds values across rows. ROUND() makes numeric output easier to read.

Source interpretation: AB de Villiers strikes at 209.83 chasing at the death, from 295 balls. Two conditions at once, and the shortlist gets short fast.

SQL Query:

```
SELECT batter, -- what to show
SUM(is_legal) AS balls, -- sample size
ROUND(100.0*SUM(batter_runs)/SUM(is_legal),2) -- runs per 100 bal
AS sr
FROM v_ball -- one row = one ba
WHERE innings=2 AND phase='Death' -- death overs only
GROUP BY batter -- one row per group
HAVING balls >= 200 -- filter the groups
ORDER BY sr DESC -- sort
LIMIT 5;
```

M5 — Find a spinner who can bowl in the powerplay without leaking runs.

Explanation: 4. What the important SQL pieces are doing HAVING filters groups after aggregation. LIMIT restricts the number of rows displayed. LIKE searches text patterns; % means any number of characters. SUM() adds values across rows. ROUND() makes numeric output easier to read.

Source interpretation: Iqbal Abdulla goes at 6.33 in the powerplay from 276 balls. A spinner who can open the bowling is a genuine tactical weapon.

SQL Query:

```
SELECT bowler, -- what to show
SUM(is_legal) AS balls, -- sample size

IPL CRICKET ANALYTICS • STUDENT SQL BOOKLET
Page 76
ROUND(6.0*SUM(bowler_runs)/SUM(is_legal),2) -- runs per over conc
AS economy
FROM v_ball
WHERE phase='Powerplay' AND (bowler_type
LIKE '%Legbreak%' OR bowler_type LIKE
'%Offbreak%' OR bowler_type LIKE
'%Orthodox%' OR bowler_type LIKE
'%Googly%' OR bowler_type LIKE
'%Wrist spin%')
GROUP BY bowler
HAVING balls >= 200 -- one row = one ball -- powerplay only
-- one row per group -- filter the groups
ORDER BY economy ASC -- sort
LIMIT 5;
```

M6 — Which batter should face the most balls at this ground?

Explanation: 4. What the important SQL pieces are doing HAVING filters groups after aggregation. LIMIT restricts the number of rows displayed. SUM() adds values across rows. ROUND() makes numeric output easier to read. JOIN connects related rows from two sources using a matching key.

Source interpretation: At Wankhede specifically, GJ Maxwell strikes at 176.33 from 207 balls. Ground-specific form is a real recruitment input.

SQL Query:

```
SELECT b.batter, -- what to show
SUM(b.is_legal) AS balls,
ROUND(100.0*SUM(b.batter_runs)/SUM(b.is_legal),2) -- round it off
AS sr_here
FROM v_ball b -- one row = on
JOIN matches_clean m ON -- attach the o
m.match_id=b.match_id
WHERE m.venue_clean='Wankhede Stadium' -- these rows o
GROUP BY b.batter -- one row per
HAVING balls >= 200 -- filter the g
ORDER BY sr_here DESC -- sort
LIMIT 5;
```

M7 — Build a shortlist: middle-overs batters with a strike rate above 140

Explanation: 4. What the important SQL pieces are doing HAVING filters groups after aggregation. LIMIT restricts the number of rows displayed. CASE WHEN creates a value using if/then/else logic. SUM() adds values across rows. ROUND() makes numeric output easier to read.

Source interpretation: V Sehwag, RM Patidar, N Pooran and H Klaasen all clear both bars. This is a shortlist a coach could actually work from.

SQL Query:

```
SELECT b.batter, -- what to show
       SUM(b.batter_runs) AS middle_runs,
       ROUND(100.0*SUM(b.batter_runs)/SUM(b.is_legal),2) -- round it off
       AS sr,
       SUM(CASE WHEN b.player_out = b.batter
              THEN 1 ELSE 0 END) AS -- branches of
outs_in_middle,
       ROUND(1.0*SUM(b.batter_runs)/ -- round it off
NULLIF(SUM(CASE WHEN
b.player_out = b.batter THEN 1
ELSE 0 END),0),2) AS average -- branches of
FROM v_ball b -- one row = on
WHERE b.phase = 'Middle' -- middle overs
GROUP BY b.batter -- one row per
HAVING SUM(b.is_legal) >= 500 AND sr >= 140 -- filter the g
AND average >= 25 -- and this too
ORDER BY sr DESC -- sort
LIMIT 8;
```

M8 — Which players do well in both innings, rather than only one?

Explanation: 4. What the important SQL pieces are doing HAVING filters groups after aggregation. LIMIT restricts the number of rows displayed. CASE WHEN creates a value using if/then/else logic. SUM() adds values across rows. ROUND() makes numeric output easier to read.

Source interpretation: V Sehwag strikes at 149.4 batting first and 164.2 chasing. A player who performs in both innings is rarer than the leaderboards suggest.

SQL Query:

```
SELECT batter, -- what to show
       ROUND(100.0*SUM(CASE WHEN innings=1 -- round it off
THEN batter_runs ELSE 0 END) -- branches of the CASE
/NULLIF(SUM(CASE WHEN -- divide, no zero risk
innings=1 THEN is_legal ELSE 0
END),0),1) AS sr_first, -- branches of the CASE
       ROUND(100.0*SUM(CASE WHEN innings=2 -- round it off
THEN batter_runs ELSE 0 END) -- branches of the CASE
/NULLIF(SUM(CASE WHEN -- divide, no zero risk
innings=2 THEN is_legal ELSE 0
END),0),1) AS sr_second -- branches of the CASE
FROM v_ball -- one row = one ball
GROUP BY batter -- one row per group
HAVING SUM(CASE WHEN innings=1 THEN is_legal -- filter the groups
ELSE 0 END) >= 800 AND SUM(CASE WHEN -- branches of the CASE
innings=2 THEN is_legal ELSE 0 END)
>= 800 AND sr_first >= 135 AND
sr_second >= 135
ORDER BY (sr_first + sr_second) DESC -- sort
LIMIT 5;
```

M9 — Rank every batter within their own team by runs scored.

Explanation: 4. What the important SQL pieces are doing GROUP BY creates one result row for each group. ORDER BY controls the final sorting. LIMIT restricts the number of rows displayed. SUM() adds values across rows. RANK() assigns ranks and gives equal values the same rank.

Source interpretation: Within Chennai, MS Dhoni leads on 4,865 runs and SK Raina follows on 4,687. RANK inside a team is how squad reports are actually written.

SQL Query:

```
SELECT batting_team, -- name a first query
       batter, SUM(batter_runs) AS runs FROM -- total runs
v_ball GROUP BY batting_team, batter)
SELECT batting_team, -- what to show
       batter,
       runs,
       RANK() OVER (PARTITION BY -- rank within group
batting_team ORDER BY runs
DESC) AS rank_in_team
FROM totals -- the source table
WHERE batting_team = 'Chennai Super Kings' -- filter the rows
ORDER BY rank_in_team -- sort
LIMIT 5;
```